

Hibernate Introduction

Hibernate is a powerful Java framework that simplifies database interactions by providing an Object-Relational Mapping (ORM) solution. It maps Java objects to database tables, allowing developers to work with data in an object-oriented manner without writing complex SQL queries.

Prerequisites for Learning Hibernate

1. *Java:*

- Strong understanding of Java fundamentals, including OOP concepts.
- Familiarity with Java Collections, Exceptions, and Interfaces.

2. *DBMS (Database Management Systems):*

- Basic knowledge of relational databases (e.g., MySQL, PostgreSQL).
- Understanding how data is stored, retrieved, and manipulated.

3. *SQL (Structured Query Language):*

- Ability to write and understand basic to intermediate SQL queries.

4. *JDBC (Java Database Connectivity):*

- Knowledge of setting up database connections in Java.
- Understanding how to execute SQL queries and process results using JDBC.

Fundamentals Before Hibernate

Persistence:

- **Definition:** The process of storing and managing data for a long time is called persistence.
- **Examples:**
 1. **File Operations:** Storing data in files on a hard disk (e.g., using Java I/O operations).
 2. **Database Storage:** Storing data in the form of tables in a database (e.g., using SQL).

Persistence Store:

- **Definition:** A persistence store is a place or storage where data is saved and managed for an extended period.

Persistent Data:

- **Definition:** The data stored in a persistence store is referred to as persistence data.

Persistence Operations:

- **Definition:** The operations performed on persistent data, such as creating, reading, updating, or deleting data.

Persistence Logic:

- **Definition:** The logic written to perform persistence operations is called persistence logic. This logic ensures data is correctly stored and retrieved.

Persistence Technology/Framework:

- **Definition:** The technology or framework used to write persistence logic is known as persistence technology or framework.
- **examples:**
 - **JDBC (Java Database Connectivity):** A technology for connecting and executing queries against the database.

- **Hibernate:** An ORM (Object-Relational Mapping) framework that simplifies database interactions by mapping Java objects to database tables.
- **Spring JDBC / Spring ORM / Spring Data JPA:** Frameworks provided by Spring to manage persistence logic with advanced features and integration.

Additional Notes:

- ***Persistence in Modern Applications:*** Modern applications often require persistent data storage to ensure that data is not lost when the application is closed or the system is restarted. Technologies like Hibernate abstract the complexity of database interactions, allowing developers to focus on business logic.
- ***Importance of ORM:*** ORM frameworks like Hibernate automate many of the tedious aspects of database management, such as writing SQL queries and managing connections, which can lead to faster development and more maintainable code.

Limitations of JDBC (Java Database Connectivity)

1. SQL Query Language Dependency:

- Developers must manually write and manage SQL queries for all database operations.
- This increases the complexity and potential for errors, especially in large applications with complex queries.

2. Repetitive Code:

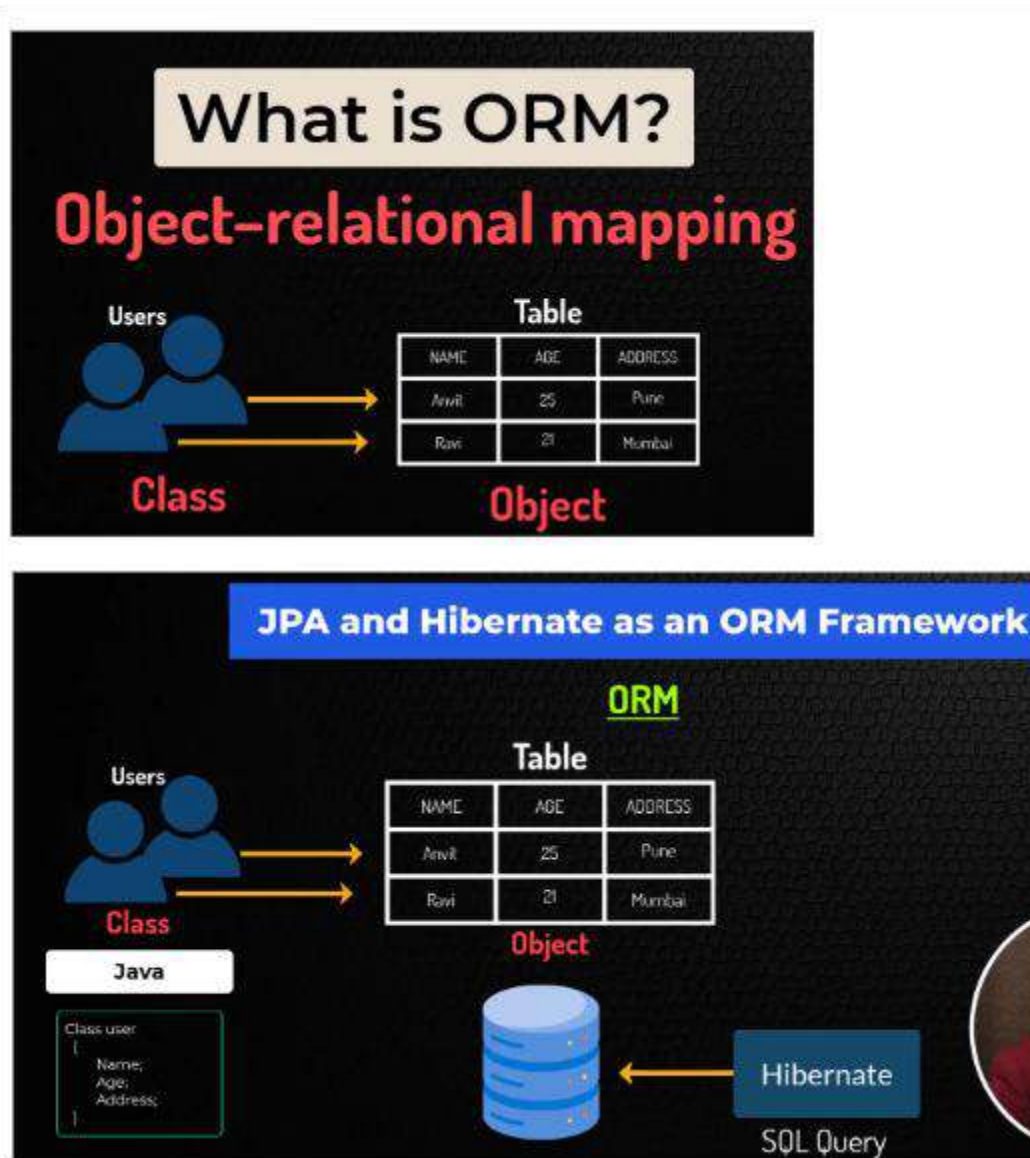
- JDBC requires boilerplate code for tasks such as establishing connections, creating statements, and handling exceptions.
- The repetitive nature of this code can lead to increased development time and maintenance challenges.

3. Lack of Named Parameters:

- JDBC only supports positional parameters in prepared statements (e.g., ? placeholders), making it harder to manage queries with multiple parameters.
- Named parameters, which allow parameters to be identified by name rather than position, are not supported, reducing the readability and flexibility of queries.

These limitations often lead developers to seek out higher-level frameworks like Hibernate, which abstract away much of the complexity involved in database interactions and provide more features such as named parameters and automatic SQL generation.

JPA and Hibernate as ORM Framework



What is ORM (Object-Relational Mapping)?

- **Definition:** ORM is a technique that allows developers to map Java objects to database tables, making it easier to work with databases in an object-oriented manner.
- **How It Works:**
 - Java classes (representing entities) are mapped to database tables.
 - Each instance of a class corresponds to a row in the table.
 - This mapping allows developers to manipulate database records as if they were working with simple Java objects.

JPA (Java Persistence API):

- **Definition:** JPA is a specification provided by Java for managing relational data in Java applications.
- **Key Points:**
 - **Object-Relational Mapping:** It defines how to map Java objects to database tables.
 - **Implementations:** Hibernate and TopLink are popular implementations of the JPA specification.

Hibernate:

- **Definition:** Hibernate is a popular ORM framework that implements the JPA specification, providing a powerful and flexible way to work with relational databases in Java.
- **How It Works:**
 - Hibernate automatically generates SQL queries based on the mapping defined between Java classes and database tables.
 - These queries are then executed using JDBC (Java Database Connectivity) to interact with the database.

Key Benefits:

- **Automated SQL Generation:** Hibernate generates SQL queries automatically, reducing the need for manual query writing.
- **Ease of Data Manipulation:** Developers can work with database data as Java objects, simplifying code and improving maintainability.

Summary:

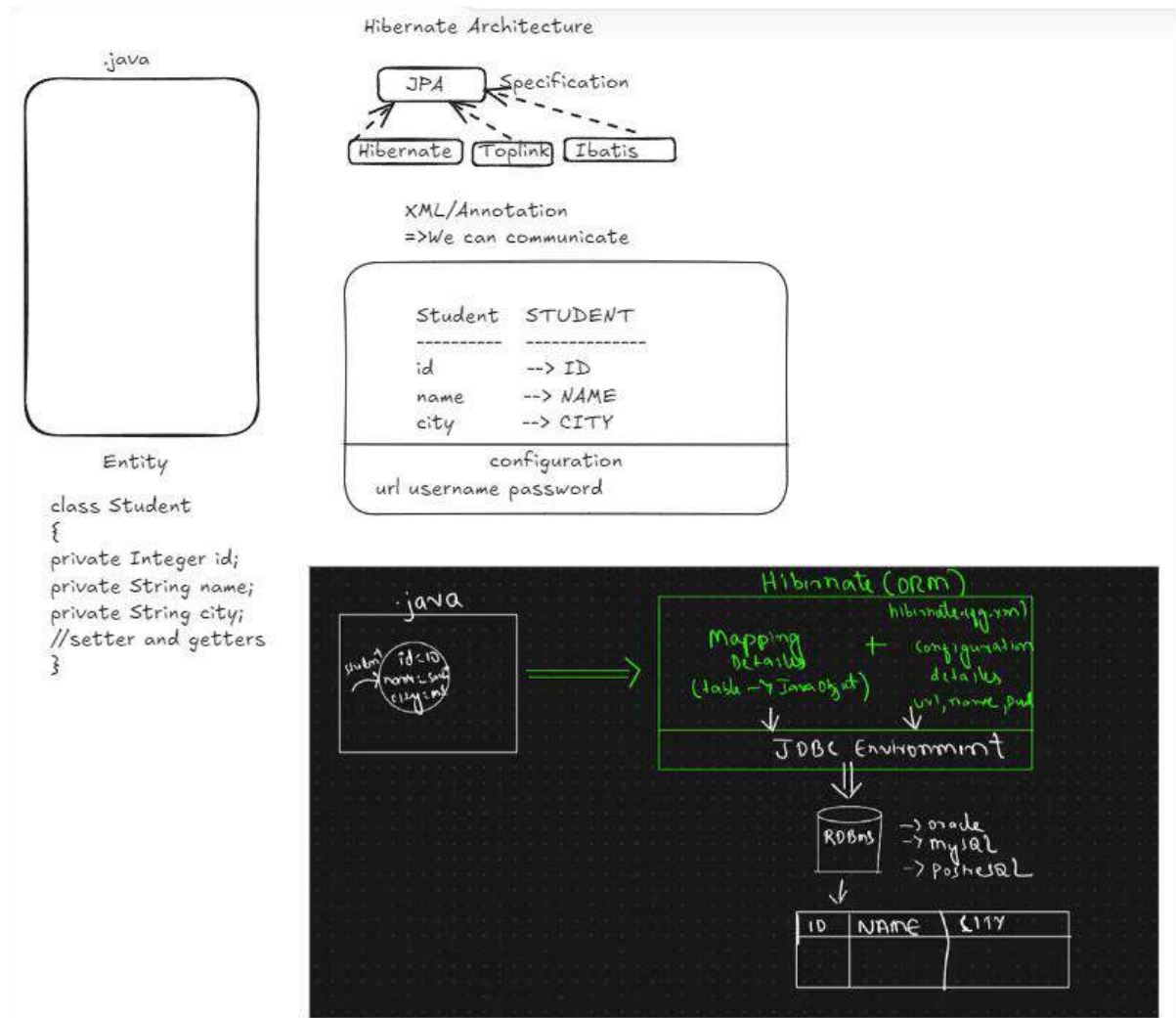
- **ORM (Object-Relational Mapping):** Maps Java classes to database tables.
- **JPA (Java Persistence API):** A specification that defines how ORM should be implemented in Java.

Hibernate: A widely-used implementation of JPA that automates database interactions, making Java development more efficient and less error-prone.

Hibernate Architecture

Hibernate is a powerful, high-performance Object-Relational Mapping (ORM) tool that simplifies the interaction between Java applications and relational databases. It abstracts the database operations by allowing developers to focus on the business logic rather than SQL queries.

Hibernate Architecture image



Core Components of Hibernate Architecture:

1. JPA (Java Persistence API) Specification:

- Hibernate implements the JPA specification, which standardizes ORM in Java.

- It provides a common interface for different ORM tools like Hibernate, Toplink, and iBatis.

2. Mapping Configuration:

- Hibernate uses XML files or annotations to map Java objects to database tables.
- Each Java class that needs to be persisted in the database is known as an **Entity**.

```
class Student {  
  
    private Integer id;  
  
    private String name;  
  
    private String city;  
  
    // Getters and setters  
  
}
```

The class Student is mapped to the STUDENT table in the database with fields id, name, and city corresponding to the columns ID, NAME, and CITY.

Hibernate Configuration:

- Hibernate configuration involves setting up the database connection details such as the URL, username, and password.
- This configuration can be done through an XML file (hibernate.cfg.xml) or programmatically via annotations.

Hibernate ORM (Object-Relational Mapping):

- Hibernate converts Java objects into database records (rows in tables) and vice versa.
- The process of converting a Java object into a database record is called **Mapping**. This involves:

- **Mapping Details:** Java objects are mapped to relational database tables.
- **Configuration Details:** These include the database connection properties and other settings needed for the ORM tool to function correctly.
- Hibernate interacts with the **JDBC Environment** to execute SQL queries and manage database transactions.

Database Connectivity:

- Hibernate can work with various relational databases (RDBMS) like Oracle, MySQL, PostgreSQL, etc.
- The ORM tool abstracts the underlying database operations, allowing developers to switch between different databases with minimal configuration changes.

Entity & Table Mapping:

- The entities (e.g., Student) in Hibernate are directly mapped to corresponding database tables (STUDENT).
- Each field in the entity class (e.g., id, name, city) is mapped to a column in the database table.

Process Flow in Hibernate:

1. Java Object Creation:

- A Java object (e.g., Student) is created with data.

2. ORM Handling:

- Hibernate ORM handles the conversion of this object into a database record by using the mapping configuration.

3. Database Operation:

- Hibernate interacts with the JDBC environment to insert, update, or query the database.

- The data is stored in the appropriate table (e.g., STUDENT table) in the chosen RDBMS.

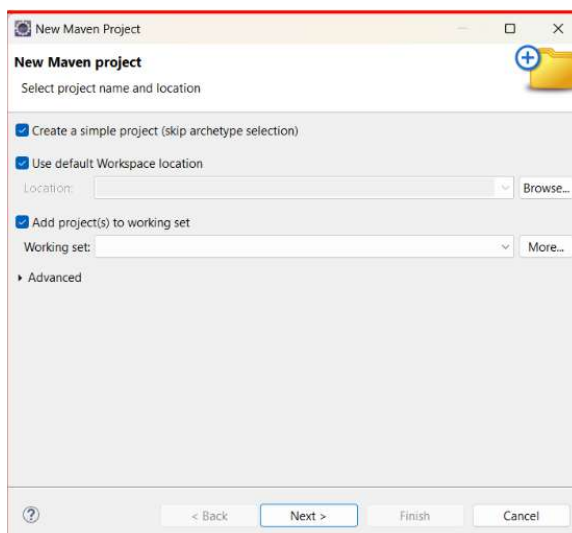
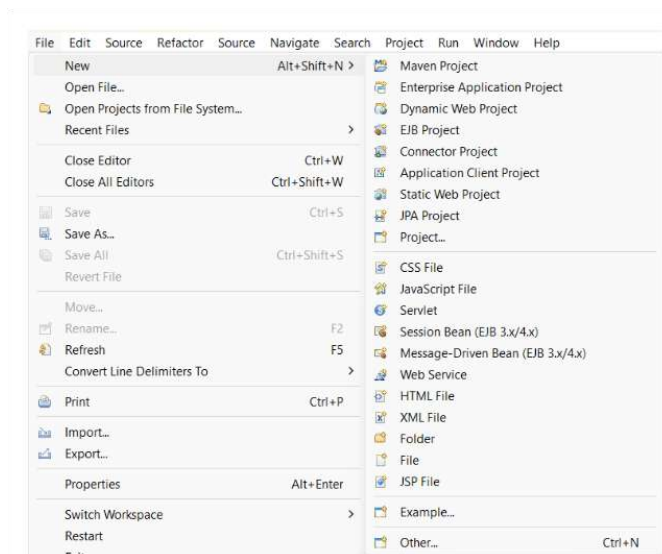
Hibernate Project Setup

What have we learned?

- . How to set up a hibernate project?
- . How to change the default JRE library?

Steps to Create a Hibernate Project

- **Step 1:** Create a Maven project using the quickstart template.



- **Step 2:** Add the Group ID, Artifact ID, and Project Name.

New Maven Project

New Maven project

Configure project

Artifact

Group Id: com.telusko

Artifact Id: HibernateFirst

Version: 0.0.1-SNAPSHOT

Packaging: jar

Name: HibernateFirst

Description:

Parent Project

Group Id:

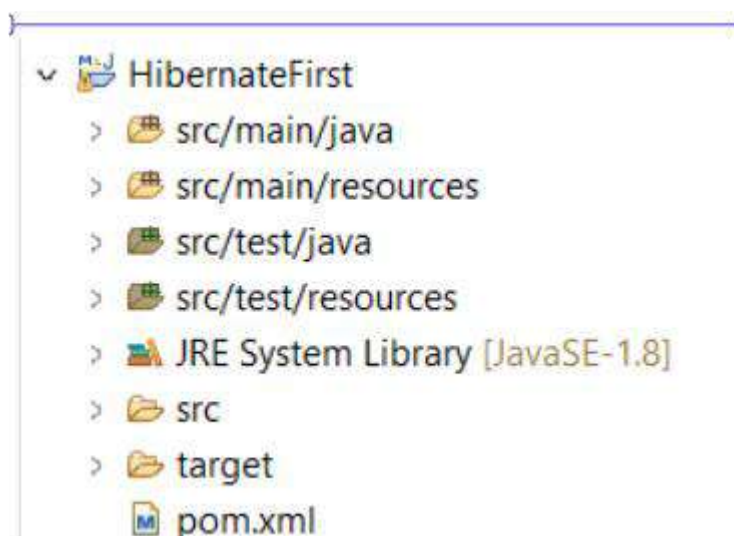
Artifact Id:

Version: Browse... Clear

Advanced

< Back Next > Finish Cancel

- **Step 3:** Open pom.xml and add the hibernate-core dependency.



Add dependency:

```
<!--  
https://mvnrepository.com/artifact/org.hibernate/hibernate-  
core -->
```

```
<dependency>  
  
    <groupId>org.hibernate</groupId>  
  
    <artifactId>hibernate-core</artifactId>  
  
    <version>6.5.2.Final</version>  
  
    <type>pom</type>  
  
</dependency>
```

pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"  
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0  
https://maven.apache.org/xsd/maven-4.0.0.xsd">  
  
    <modelVersion>4.0.0</modelVersion>  
  
    <groupId>com.telusko</groupId>  
  
    <artifactId>HibernateFirst</artifactId>  
  
    <version>0.0.1-SNAPSHOT</version>  
  
    <name>HibernateFirst</name>  
  
    <dependencies>  
  
        <!--  
https://mvnrepository.com/artifact/org.hibernate/hibernate-  
core -->  
  
        <dependency>  
  
            <groupId>org.hibernate</groupId>  
  
            <artifactId>hibernate-core</artifactId>
```

```
<version>6.5.2.Final</version>

<type>pom</type>

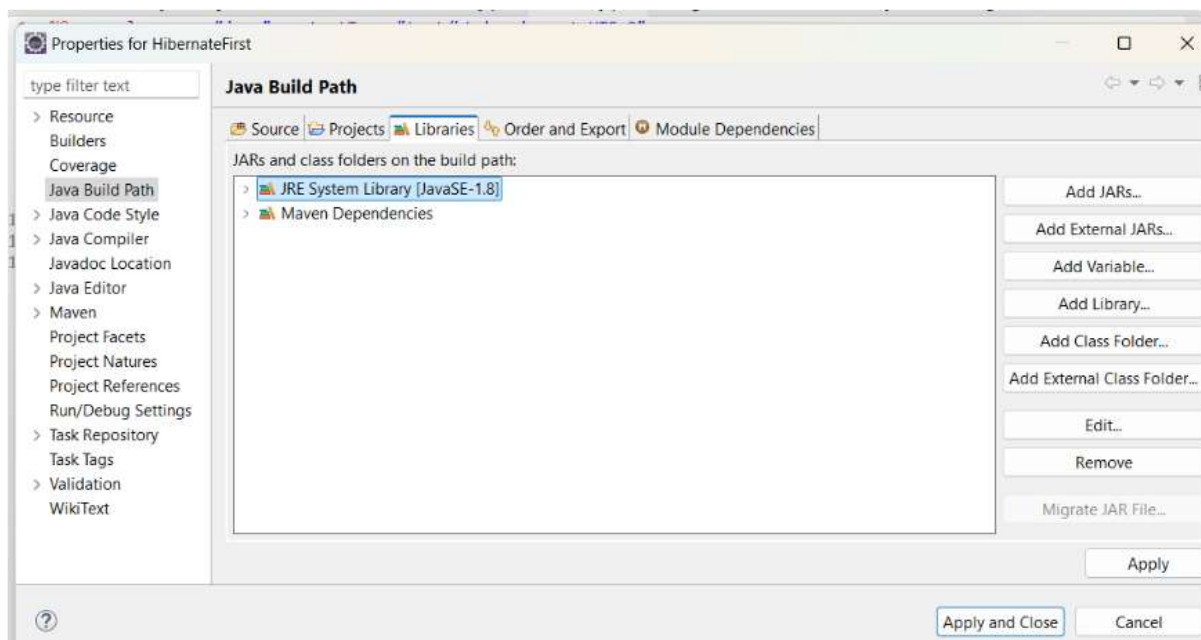
</dependency>

</dependencies>

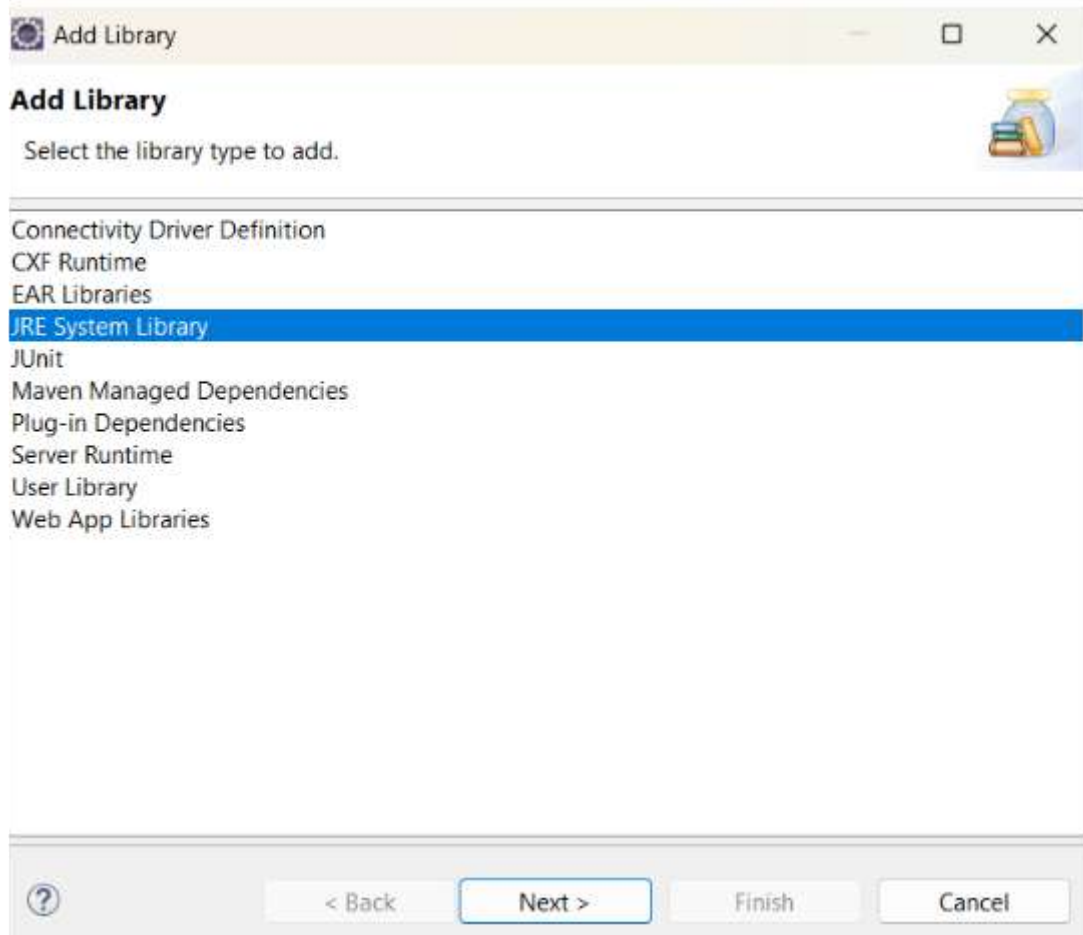
</project>
```

Changing the Default JRE Library

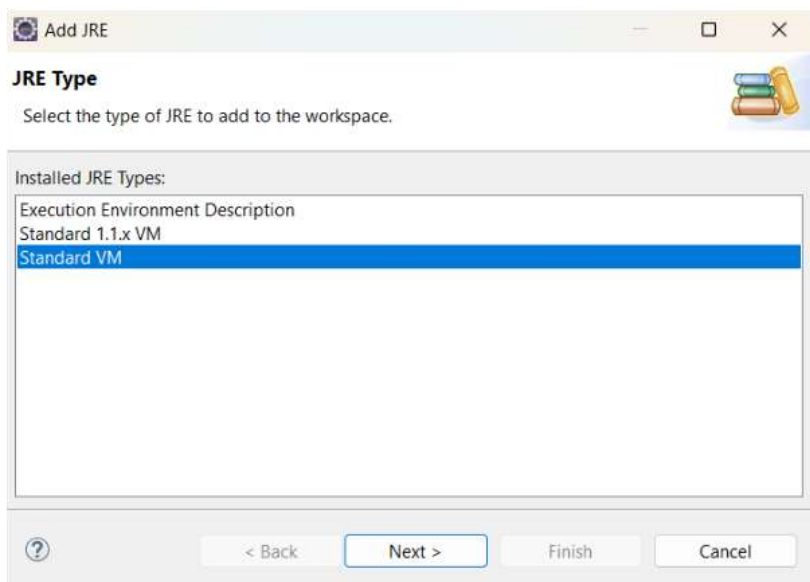
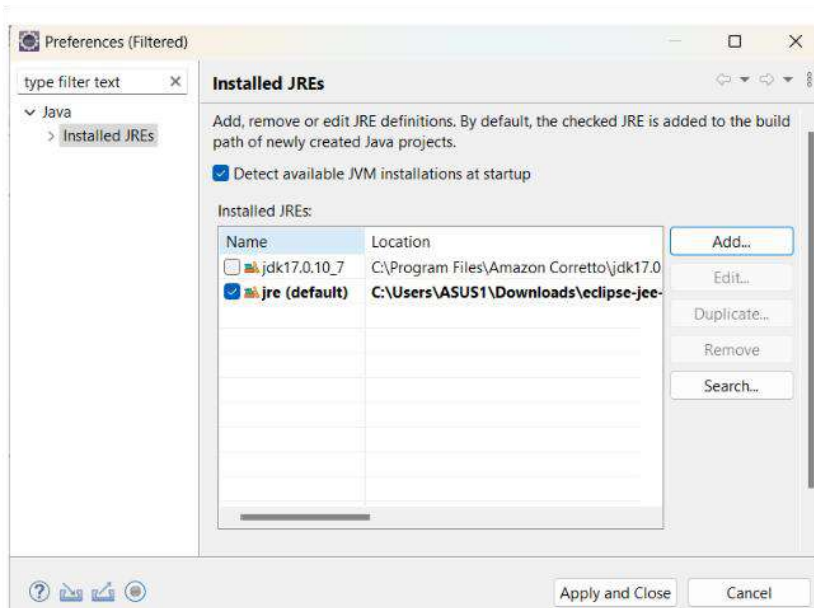
- **Step 1:** Open the Java build path settings.

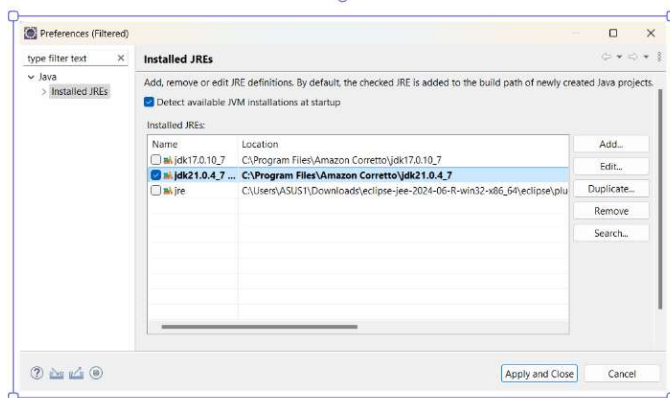
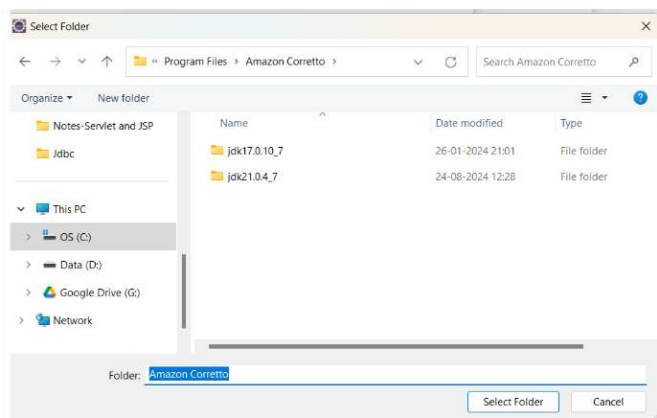
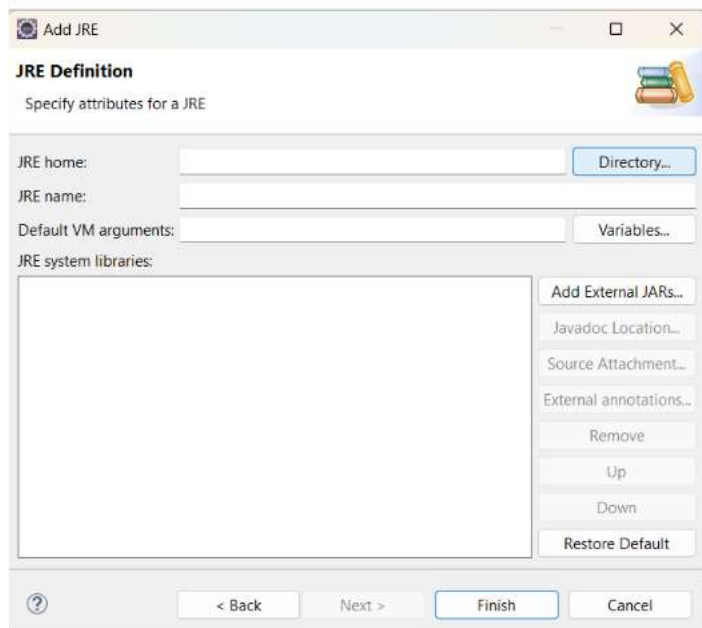


- **Step 2:** Remove the default JRE library.
- **Step 3:** Click "Add Library" and select "JRE System Library."



- **Step 4:** Choose the JDK directory where your installed JDK is located (e.g., JDK 21 or 17).

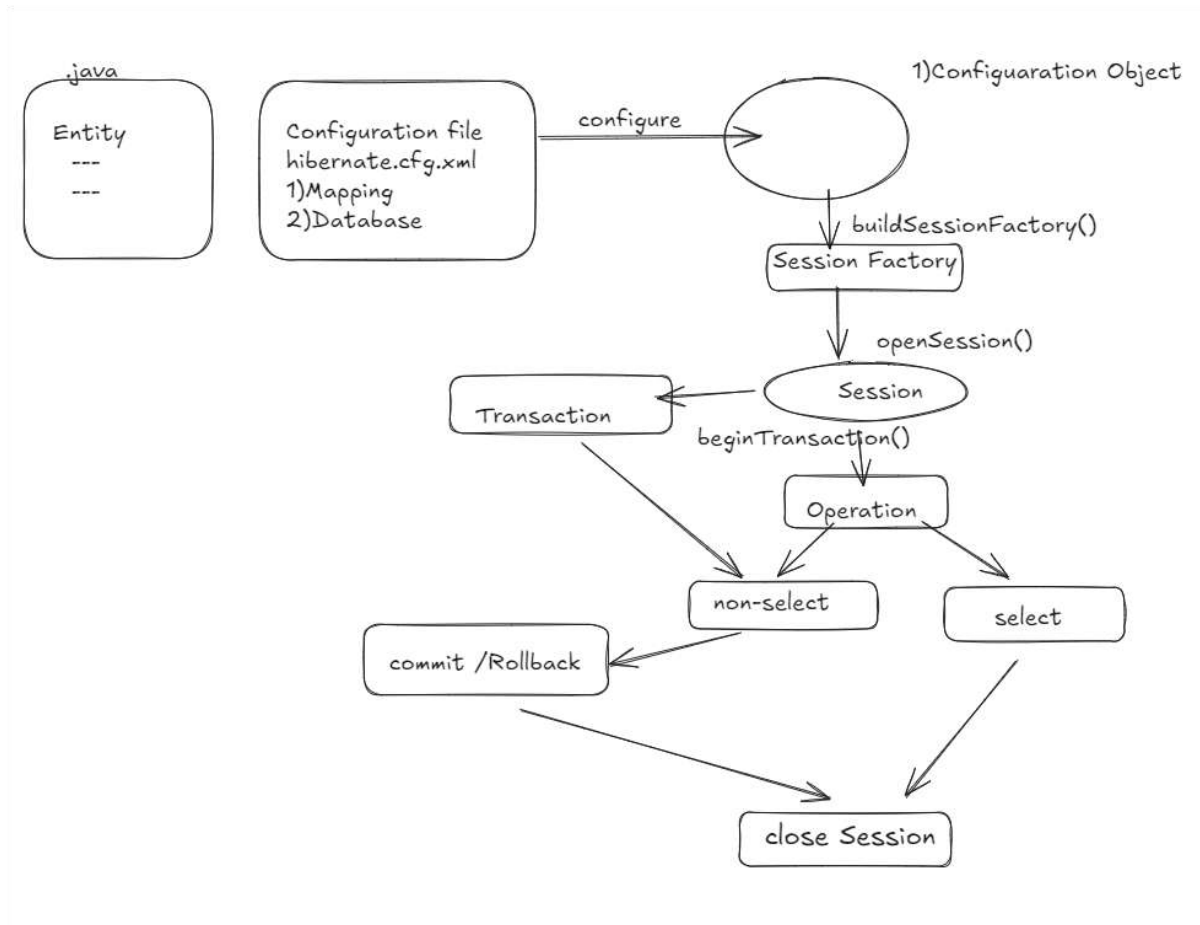




- **Step 5:** Apply the changes and finish.

- src/main/java
 - src/main/resources
 - src/test/java
 - src/test/resources
 - Maven Dependencies
 - jakarta.persistence-api-3.1.0.jar - C:\Users\ASUS1\m 1
 - jakarta.transaction-api-2.0.1.jar - C:\Users\ASUS1\m 1
 - jboss-logging-3.5.0.Final.jar - C:\Users\ASUS1\m2\re 1
 - hibernate-commons-annotations-6.0.6.Final.jar - C:\ 1
 - jandex-3.1.2.jar - C:\Users\ASUS1\m2\repository\io\ 1
 - classmate-1.5.1.jar - C:\Users\ASUS1\m2\repository\ 1
 - byte-buddy-1.14.15.jar - C:\Users\ASUS1\m2\reposit 1
 - jakarta.xml.bind-api-4.0.0.jar - C:\Users\ASUS1\m2\ 1
 - jakarta.activation-api-2.1.0.jar - C:\Users\ASUS1\m2 1
 - jaxb-runtime-4.0.2.jar - C:\Users\ASUS1\m2\reposito 1
 - jaxb-core-4.0.2.jar - C:\Users\ASUS1\m2\repository\ 1
 - angus-activation-2.0.0.jar - C:\Users\ASUS1\m2\repi
 - txw2-4.0.2.jar - C:\Users\ASUS1\m2\repository\org\
 - istack-commons-runtime-4.1.1.jar - C:\Users\ASUS1\
 - jakarta.inject-api-2.0.1.jar - C:\Users\ASUS1\m2\repi
 - antlr4-runtime-4.13.0.jar - C:\Users\ASUS1\m2\repo
 - JRE System Library [jdk21.0.4_7]
 - src
 - target
 - pom.xml

Hibernate Project Setup Architecture



Note: No transaction needed for select operation.

1. Entity Class (Java):

- Represents the Java class that needs to be persisted in the database.
- This class is mapped to a database table via configuration.

2. Configuration File (hibernate.cfg.xml):

- The central configuration file for Hibernate.
- It includes details for:
 - **Mapping:** Specifies how Java classes are mapped to database tables.
 - **Database Configuration:** Contains information like the database URL, username, and password.

3. Configuration Object:

- Loads the Hibernate configuration file.
- It initializes Hibernate using the configure() method.

4. SessionFactory:

- Created by the buildSessionFactory() method.
- It is a factory for Session objects.
- Designed to be instantiated once and used throughout the application lifecycle.

5. Session:

- The primary interface for interacting with the database.
- Obtained from the SessionFactory using openSession().
- Used to create transactions and perform database operations.

6. Transaction:

- Represents a unit of work in Hibernate.
- Managed through the beginTransaction() method.
- Operations can be either:
 - **Non-Select Operations:** Involve insert, update, or delete actions.
 - **Select Operations:** Involve querying the database to retrieve data. (Note: does not required for select query)

7. Commit/Rollback:

- After the operations are performed, the transaction must be either committed or rolled back.
- **Commit:** Saves the changes to the database.
- **Rollback:** Reverts the changes in case of an error or cancellation.

8. Close Session:

- The session should be closed after completing the operations.
- This releases the database connection and other resources.

Install the Helper plugin in Eclipse IDE

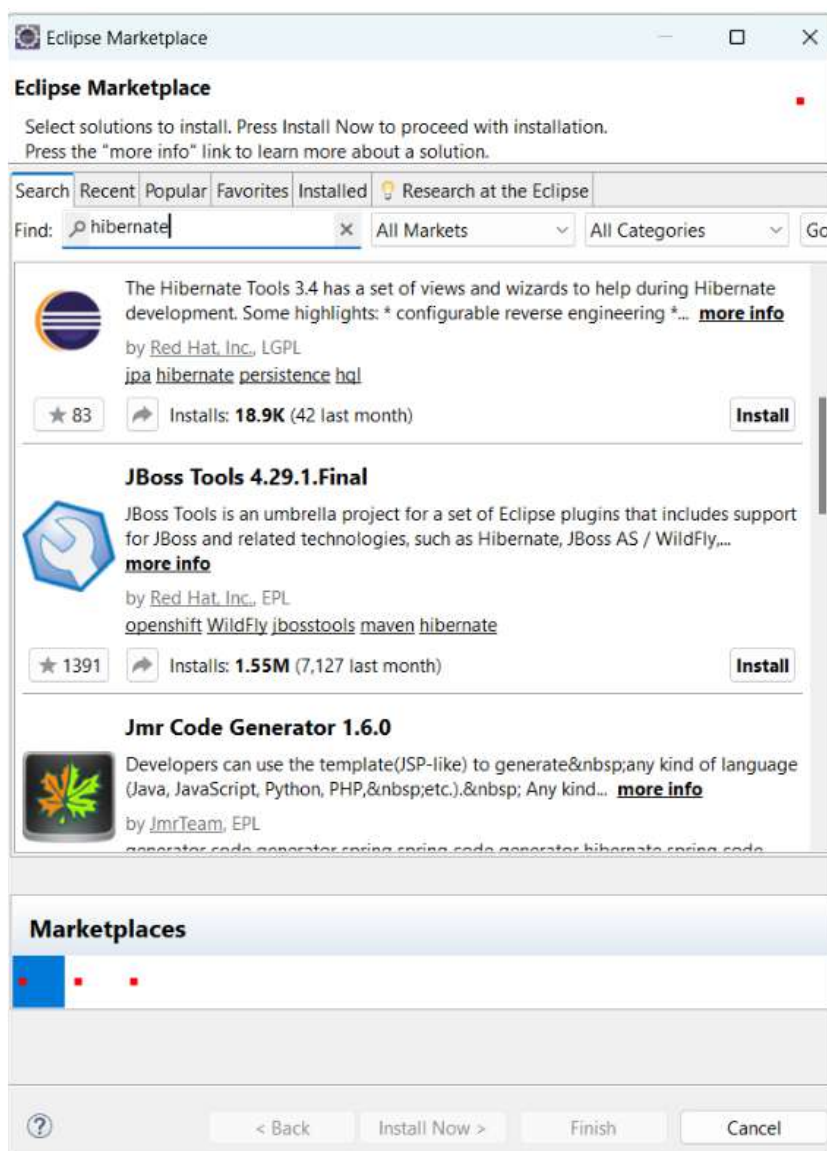
Need of JBOSS plugin (specific for Eclipse IDE)

The JBoss Tools plugin for Hibernate simplifies development by generating configuration files, entity classes, and CRUD operations automatically. It includes an HQL editor, schema visualization, and seamless Eclipse integration, saving time and enhancing productivity.

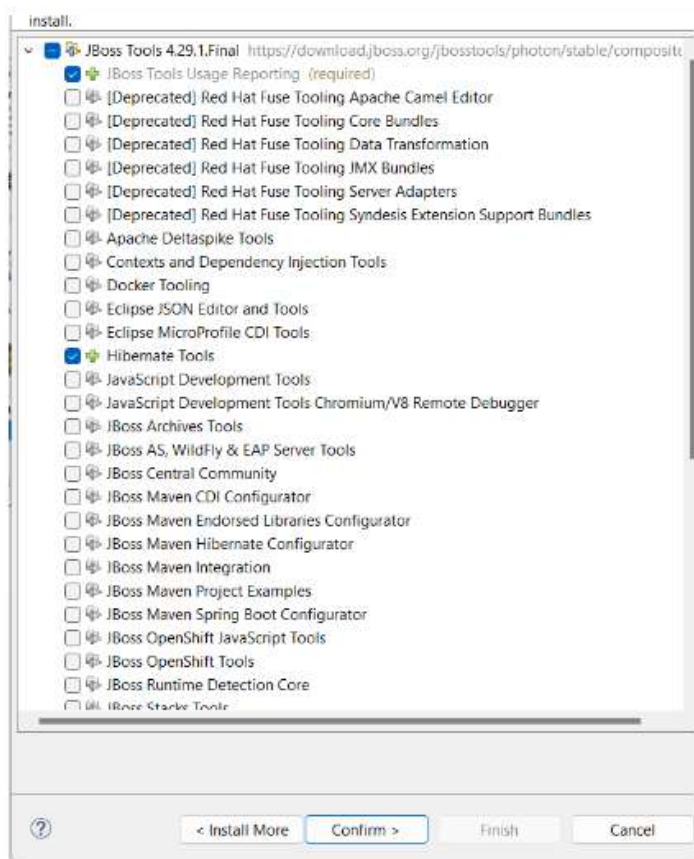
Steps to install the helper plugin (JBOSS) for Hibernate:

Step1: Click on Help and then select Eclipse Marketplace.

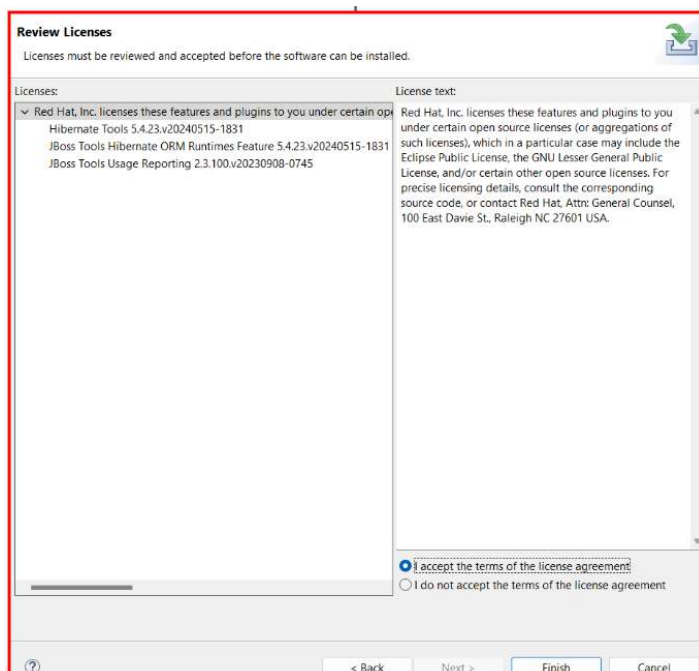
Choose Jboss



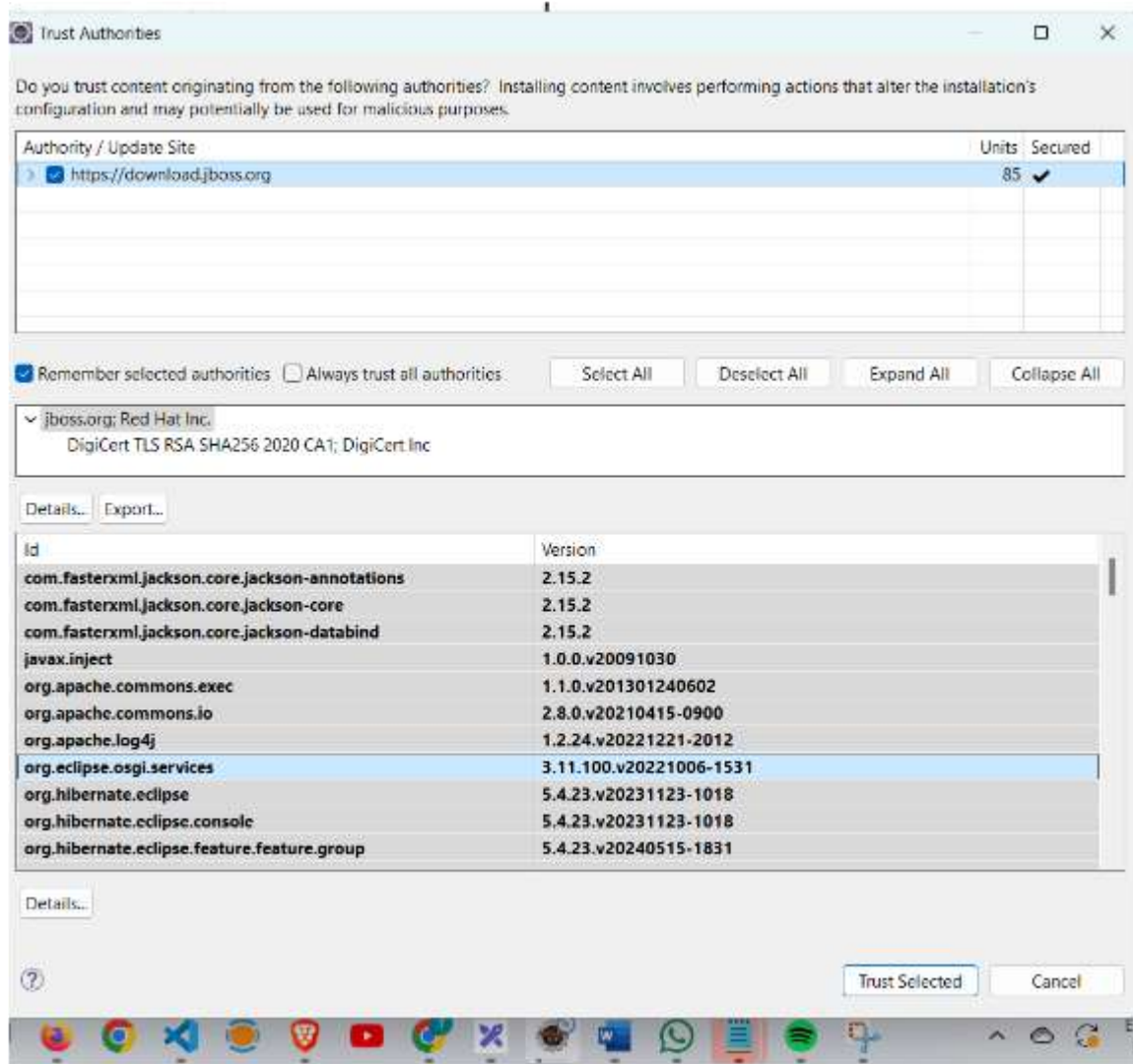
Step 2: Select only Hibernate tools.



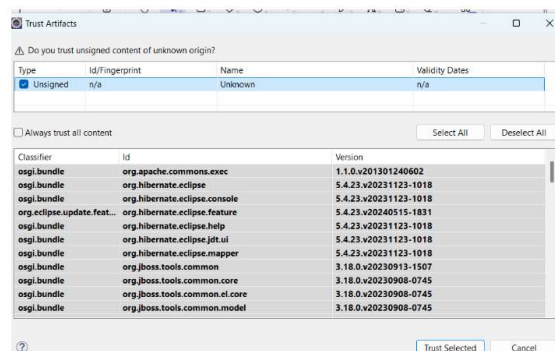
Step3: Accept license



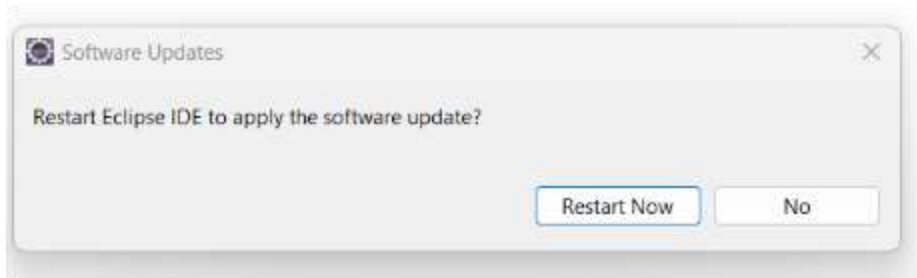
Step(optional): If you get such things, just **check the** check option and then click on **Trust selected**.



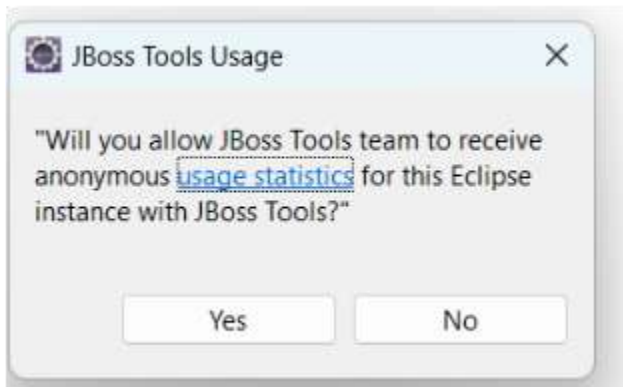
Step(Optional): If you get such things, just **check the** check option and then click on **Trust selected**.



Step(final): Restart your eclipse



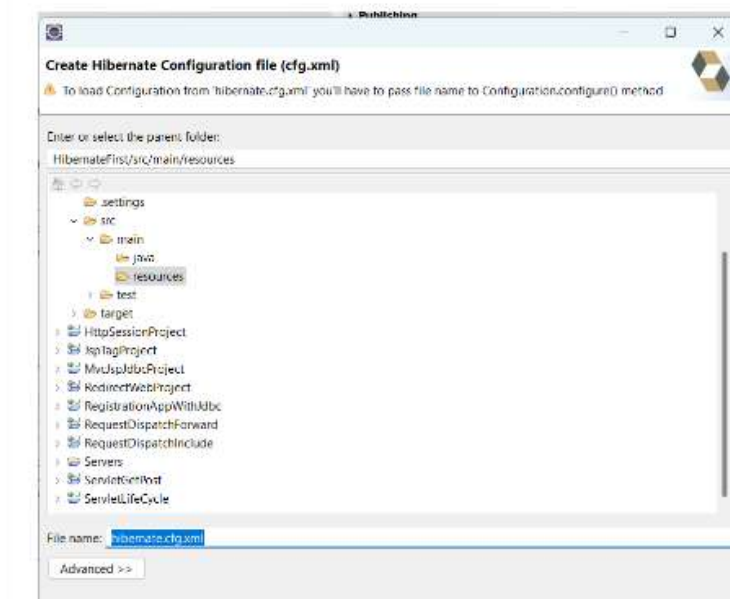
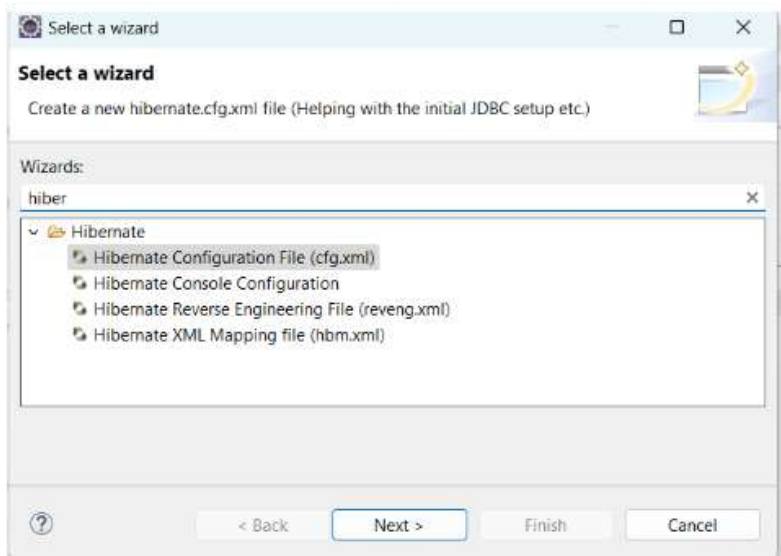
Step(Optional): If you get after restarting then select Yes.



Create a hibernate.cfg.xml file using a plugin.

Step1: Search hibernate and choose cfg.xml file

Create the name of the XML file by (hibernate.cfg.xml)



Step2: Enter details of configurations

Hibernate Configuration File (cfg.xml)

This wizard creates a new configuration file to use with Hibernate.

Container: /HibernateFirst/src/main/resources

File name: hibernate.cfg.xml

Hibernate version: 6.5

Session factory name:

[Get values from Connection](#)

Database dialect: org.hibernate.dialect.MySQL8Dialect

Driver class: com.mysql.cj.jdbc.Driver

Connection URL: jdbc:mysql://localhost:3306/telusko_db

Default Schema:

Default Catalog:

Username: root

Password: root

☐ Create a console configuration

< Back Next > Finish Cancel

Final file:

```
--//Hibernate/Hibernate Configuration DTD 3.0//EN (doctype with catalog)
1 <?xml version="1.0" encoding="UTF-8"?>
2 <!DOCTYPE hibernate-configuration PUBLIC
3   "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
4   "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
5 <hibernate-configuration>
6   <session-factory>
7     <property name="hibernate.connection.driver_class">com.mysql.cj.jdbc.Driver</property>
8     <property name="hibernate.connection.password">root</property>
9     <property name="hibernate.connection.url">jdbc:mysql://localhost:3306/telusko_db</property>
10    <property name="hibernate.connection.username">root</property>
11    <property name="hibernate.dialect">org.hibernate.dialect.MySQL8Dialect</property>
12  </session-factory>
13 </hibernate-configuration>
14
```

First Hibernate Application

Hibernate is a popular ORM (Object-Relational Mapping) framework that simplifies database interactions by mapping Java classes to database tables. Below are the key components and steps for creating your first Hibernate application.

1. Model Class: Student

- **Annotations Used:**

- `@Entity`: Marks the class as an entity, meaning it will be mapped to a database table.
- `@Table(name = "StudentTable")`: Specifies the name of the database table.
- `@Id`: Marks the primary key of the entity.
- `@Column(name = "SID")`: Maps the field to the corresponding column in the database.

- **Code Example:**

```
@Entity
@Table(name="StudentTable")
public class Student {

    @Id
    @Column(name="SID")
    private Integer sid;

    @Column(name="SNAME")
    private String sname;

    @Column(name="SCITY")
    private String scity;

    public Student() {
        System.out.println("Zero Param Constructor for Hibernate");
    }
}
```

```
}  
  
    // Getters and Setters  
  
}
```

Notes: If you do not specify the table name or column names in Hibernate using the `@Table` and `@Column` annotations, Hibernate will apply default naming conventions:

1. Default Table Name:

- By default, the table name will be the same as the class name of the entity.
- For example, if your entity class is named `Student`, the default table name will be `Student`.

2. Default Column Names:

- The column names will be the same as the field names in your entity class.
- For example, if you have a field `private String sname;`, the default column name will be `sname`.

2. Hibernate Configuration File (`hibernate.cfg.xml`)

● **Important Properties:**

- `hibernate.connection.driver_class`: Specifies the JDBC driver class.
- `hibernate.connection.url`: The JDBC URL to connect to the database.
- `hibernate.connection.username` and `hibernate.connection.password`: Credentials for the database.
- `hibernate.dialect`: Specifies the SQL dialect Hibernate should use (e.g., `org.hibernate.dialect.MySQL8Dialect`).
- `hibernate.hbm2ddl.auto`: Specifies how Hibernate should handle database schema creation/update. Values can be `create`, `update`, `validate`, etc.
- `hibernate.show_sql`: If set to `true`, Hibernate will log the generated SQL to the console.

- hibernate.format_sql: Formats the SQL output in the console for better readability.
- <mapping class="com.telusko.model.Student"/>: Specifies the class to be mapped to the database.

- **Code Example:**

```
<hibernate-configuration>
  <session-factory>
    <property
name="hibernate.connection.driver_class">com.mysql.jdbc.Driver</property>
    <property
name="hibernate.connection.url">jdbc:mysql://localhost:3306/yourDatabaseName</property>
    <property name="hibernate.connection.username">root</property>
    <property name="hibernate.connection.password">root123</property>
    <property
name="hibernate.dialect">org.hibernate.dialect.MySQL8Dialect</property>
    <property name="hibernate.hbm2ddl.auto">create</property>
    <property name="hibernate.show_sql">true</property>
    <property name="hibernate.format_sql">true</property>
    <mapping class="com.telusko.model.Student"/>
  </session-factory>
</hibernate-configuration>
```

3. Main Class for Hibernate Operations

- **Steps Involved:**

1. **Configuration Object:** Loads the Hibernate configuration file.
2. **Build SessionFactory:** Creates a SessionFactory from the configuration object.
3. **Open Session:** Opens a new session from the SessionFactory.
4. **Begin Transaction:** Begins a transaction for database operations.

5. **Perform Operations:** In this example, create a Student object, set its properties, and save it to the database.
 6. **Commit Transaction:** Commits the transaction to persist the changes.
 7. **Close Session:** Closes the session after the operation is complete.
- **Code Example:**

```
public static void main(String[] args) {  
    // Step 1: Configuration Object  
    Configuration config = new Configuration();  
    config.configure();  
  
    // Step 2: Create SessionFactory Object  
    SessionFactory sessionFactory = config.buildSessionFactory();  
  
    // Step 3: Get the Session Object from Session Factory  
    Session session = sessionFactory.openSession();  
  
    // Step 4: Begin the Transaction within Session  
    Transaction transaction = session.beginTransaction();  
  
    // Step 5: Perform Operation  
    Student student = new Student();  
    student.setSid(1233);  
    student.setSname("Arjun");  
    student.setScity("Ballia");  
  
    session.save(student);  
  
    // Step 6: Commit Transaction  
    transaction.commit();  
  
    // Step 7: Close the Session  
    session.close();  
}
```

```
}
```

Note on Configuring hibernate.cfg.xml File

- **Default Behavior:** When configuring Hibernate, the framework automatically looks for a configuration file named hibernate.cfg.xml in the classpath. You can load this configuration using:

```
Configuration config = new Configuration();  
config.configure();
```

- **Custom Configuration File Name:**
 - If your configuration file is named something other than hibernate.cfg.xml (e.g., hib.cfg.xml), you must explicitly specify this file name when configuring Hibernate.
 - Example:

```
Configuration config = new Configuration();  
config.configure("hib.cfg.xml");
```

Key Points:

- **Default:** config.configure(); assumes the file is named hibernate.cfg.xml.
- **Custom:** Use config.configure("custom-file-name.xml"); if you have renamed your configuration file.

Output:

```
mysql> use telusko_db  
Database changed  
mysql> select * from Student;  
ERROR 1146 (42S02): Table 'telusko_db.student' doesn't exist  
mysql> show tables;  
+-----+  
| Tables_in_telusko_db |  
+-----+  
| personinfo           |  
| studenttable         |  
+-----+  
2 rows in set (0.01 sec)  
  
mysql> select * from studenttable;  
+-----+  
| sid | scity | sname |  
+-----+  
| 123 | Ballia | Shiva |  
| 1233 | Ballia | Arjun |  
+-----+  
2 rows in set (0.00 sec)
```


Insert Using persist() method

In Hibernate, inserting data into the database is typically achieved using session operations like `persist()`, `save()`, or `saveOrUpdate()`. This process involves creating an entity, starting a transaction, and using the session object to persist the entity into the database.

Steps to Insert Data Using Hibernate:

- 1. Configure Hibernate:**
 - Create a Configuration object to configure Hibernate with properties and mappings.
 - Load the `hibernate.cfg.xml` file with `config.configure()`.
- 2. Create a SessionFactory and Session:**
 - SessionFactory is a factory for creating Session objects.
 - Open a session using `sessionFactory.openSession()`.
- 3. Begin a Transaction:**
 - Always begin a transaction before performing any database operation.
 - Use `session.beginTransaction()` to start a transaction.
- 4. Create an Entity Object:**
 - Instantiate the entity object (e.g., Student) and set its properties.
- 5. Insert Entity Using `session.persist()`:**
 - Use `session.persist()` to insert the entity into the database.
 - Commit the transaction using `transaction.commit()`.
- 6. Handle Exceptions:**
 - Catch exceptions such as `EntityExistsException`, `ConstraintViolationException`, and `HibernateException`.
- 7. Close Resources:**
 - Use `session.close()` and `sessionFactory.close()` in the finally block.

Exceptions during Insert Operations in Hibernate

- 1. EntityExistsException:**
 - Thrown when trying to persist an entity that already exists in the database.

```
catch (EntityExistsException e) {  
    System.out.println("Entity already exists: " + e.getMessage());  
}
```

2. ConstraintViolationException:

- Thrown when database integrity constraints (like unique constraints) are violated.

```
catch (ConstraintViolationException e) {  
    System.out.println("Constraint violation: " + e.getMessage());  
}
```

3. HibernateException:

- A general exception that indicates a problem with the Hibernate framework.

```
catch (HibernateException e) {  
    System.out.println("Hibernate exception: " + e.getMessage());  
}
```

4. Exception:

- Catches any other unexpected exceptions.

```
catch (Exception e) {  
    System.out.println("Unexpected exception: " + e.getMessage());  
}
```

persist() vs save() in Hibernate

1. persist() Method:

- **Purpose:** Used to insert an entity into the database if it does not already exist.
- **Return Type:** void (does not return the identifier).
- **Behavior:**

- Makes the given entity persistent and schedules it for insertion during flush/commit.
- Does **not** guarantee the immediate insertion.
- **Exception Handling:** Throws EntityExistsException if the entity already exists in the database.

```
session.persist(student);
```

2. save() Method:

- **Purpose:** Used to insert an entity into the database.
- **Return Type:** Returns the generated identifier (primary key) of the entity.
- **Behavior:**
 - Guarantees the immediate insertion of the entity.
 - The generated identifier is immediately available after the call.
- **Exception Handling:** Unlike persist(), save() will insert the entity, even if it is in the transient state.

```
Serializable id = session.save(student);
System.out.println("Student ID: " + id);
```

Example of save() and persist() Methods in Hibernate

Using persist():

```
Transaction transaction = session.beginTransaction();
Student student = new Student();
student.setId(134);
student.setName("Arjun");
student.setCity("Ballia");

session.persist(student); // Insert the student entity
transaction.commit();    // Commit the transaction
```

Using save():

```

Transaction transaction = session.beginTransaction();
Student student = new Student();
student.setName("Arjun");
student.setCity("Ballia");

Serializable id = session.save(student); // Insert the student entity and return the
ID
transaction.commit();
System.out.println("Student ID: " + id); // Print the generated ID

```

Key Differences Between persist() and save():

Aspect	persist()	save()
Return Type	void	Returns Serializable (primary key ID)
Entity State	Transient -> Persistent	Transient -> Persistent
Insert Guarantee	Not immediate, scheduled at flush	Immediate upon calling
Exception Handling	Throws EntityExistsException	No exception for existing entities
Recommended Usage	For standardized JPA usage	When you need the generated ID

Industry Standard Codes:

Main.java

```
import org.hibernate.cfg.Configuration;
import jakarta.persistence.EntityExistsException;
import org.hibernate.HibernateException;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.exception.ConstraintViolationException;
public class Main {
    public static void main(String[] args) {
        Configuration config = null;
        SessionFactory sessionFactory = null;
        Session session = null;
        Transaction transaction = null;
        boolean flag = false;
        try {
            config = new Configuration();
            config.configure();
            sessionFactory = config.buildSessionFactory();
            session = sessionFactory.openSession();
            Student student = new Student();
            student.setId(134);
            student.setName("Arjun");
            student.setCity("Ballia");
            transaction = session.beginTransaction();
            session.persist(student);
            transaction.commit();
            System.out.println("Transaction committed Succesfully");
            flag = true;
        } catch (EntityExistsException e) {
            System.out.println("Entity already exists: " + e.getMessage());
        } catch (ConstraintViolationException e) {
```

```

        System.out.println("Constraint violation: " + e.getMessage());
    } catch (HibernateException e) {
        System.out.println("Hibernate exception: " + e.getMessage());
    } catch (Exception e) {
        System.out.println("Unexpected exception: " + e.getMessage());
    } finally {
        if(!flag)
        {
            if (transaction != null) {
                transaction.rollback();
                System.out.println("Transaction rolled back successfully");
            }
        }
        if (session != null) {
            session.close();
        }
        if (sessionFactory != null) {
            sessionFactory.close();
        }
    }
}
}

```

Student.java

```

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name="StudentTable")
public class Student {
    @Id

```

```
@Column(name="sid")  
    private int id;
```

```
@Column(name="sname")  
    private String name;
```

```
@Column(name="scity")  
    private String city;
```

```
    public Student() {  
        System.out.println("Zero param constructor");  
    }
```

```
    public int getId() {  
        return id;  
    }
```

```
    public void setId(int id) {  
        this.id = id;  
    }
```

```
    public String getName() {  
        return name;  
    }
```

```
    public void setName(String name) {  
        this.name = name;  
    }
```

```
    public String getCity() {  
        return city;  
    }
```

```
    public void setCity(String city) {  
        this.city = city;  
    }
```

```
}
```

Result:

```
INFO: HHH10001501: Connection obtained from JdbcConnectionAccess [org.hibernate.engine.jdbc.env.internal.JdbcEnvironmentInitiator$ConnectionPr
Zero param constructor
Hibernate:
    insert
    into
        StudentTable
        (scity, sname, sid)
    values
        (?, ?, ?)
Transaction committed Successfully
Sept 07, 2024 3:36:55 PM org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl$PoolState stop
INFO: HHH10001008: Cleaning up connection pool [jdbc:mysql://localhost:3306/telusko_db]
```

```
mysql> select * from studenttable;
+-----+-----+-----+
| sid   | scity  | sname  |
+-----+-----+-----+
| 13    | Ballia | Arjun  |
| 123   | Ballia | Shiva  |
| 134   | Ballia | Arjun  |
| 1233  | Ballia | Arjun  |
| 1345  | Ballia | Shiva  |
+-----+-----+-----+
5 rows in set (0.00 sec)
```

Final Notes:

- Always handle exceptions gracefully to ensure the transaction is rolled back when necessary.
- Use `persist()` when following JPA specifications and `save()` when you need to get the generated identifier immediately.

Updating the Data

1. One-Line Configuration using `addAnnotatedClass()`:

- Replaces the need for `hibernate.cfg.xml`.

```
SessionFactory sessionFactory = new  
Configuration().configure().addAnnotatedClass(Student.class).buildSessionFactory();
```

Overall Workflow:

1. `new Configuration()`: Creates a configuration object.
2. `configure()`: Loads the Hibernate configuration details.
3. `addAnnotatedClass(Student.class)`: Registers the `Student` class as an entity.
4. `buildSessionFactory()`: Builds the `SessionFactory`, which is used to create sessions for interacting with the database.

2. `update()`:

- Used to update an existing record in the database.
- The entity must already exist in the database (identified by its ID).
- Throws an exception if the entity is not already in the database.
- Example:

```
session.update(student);
```

3. `saveOrUpdate()`:

- Saves a new entity or updates an existing one based on its identifier (ID).
- If the entity is new (no ID in the database), it inserts. If it exists, it updates.
- Example:

```
session.saveOrUpdate(student);
```

4. **merge()**:

- Similar to `saveOrUpdate()`, but works with detached entities (not in session).
- Returns a new instance that is managed by Hibernate; does not directly update the original object.
- Useful when working with multiple sessions.
- Example:

```
Student updatedStudent = session.merge(student);
```

Key Differences:

- **save()**: Only inserts new records. Throws an exception if the record exists.
- **saveOrUpdate()**: Inserts if new, updates if the record already exists.
- **merge()**: Updates detached entities and works with a new instance, does not modify the original entity directly.

Code:

```
import org.hibernate.cfg.Configuration;
import jakarta.persistence.EntityExistsException;
import org.hibernate.HibernateException;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.exception.ConstraintViolationException;
public class Main {
    public static void main(String[] args) {
        Configuration config = null;
        SessionFactory sessionFactory = null;
        Session session = null;
        Transaction transaction = null;
        boolean flag = false;
        try {
            config = new Configuration();
```

```

    config.configure();
    sessionFactory = config.buildSessionFactory();
    session = sessionFactory.openSession();
    // Detached entity that we want to update in the database
    Student student = new Student();
    student.setId(134588); // Make sure this ID exists in the database for an
update
    student.setName("Shiva Updated with Merge");
    student.setCity("New City Ballia");
    transaction = session.beginTransaction();
//    session.update(student);
//    session.saveOrUpdate(student);
    Student updatedStudent = session.merge(student); // Merge operation
    transaction.commit();
    System.out.println("Transaction committed successfully using merge.");
    flag = true;
} catch (EntityExistsException e) {
    System.out.println("Entity already exists: " + e.getMessage());
} catch (ConstraintViolationException e) {
    System.out.println("Constraint violation: " + e.getMessage());
} catch (HibernateException e) {
    System.out.println("Hibernate exception: " + e.getMessage());
} catch (Exception e) {
    System.out.println("Unexpected exception: " + e.getMessage());
} finally {
    if (!flag) {
        if (transaction != null) {
            transaction.rollback();
            System.out.println("Transaction rolled back successfully");
        }
    }
    if (session != null) {
        session.close();
    }
}

```

```

        if (sessionFactory != null) {
            sessionFactory.close();
        }
    }
}
}

```

Result:

Zero param constructor

Hibernate:

```

select
    s1_0.sid,
    s1_0.scity,
    s1_0.sname
from
    StudentTable s1_0
where
    s1_0.sid=?

```

Zero param constructor

Transaction committed successfully using merge.

```

mysql> select * from studenttable;
+-----+-----+-----+
| sid   | scity          | sname          |
+-----+-----+-----+
| 13    | Ballia         | Arjun          |
| 123   | Ballia         | Shiva          |
| 134   | Ballia         | Arjun          |
| 1233  | Ballia         | Arjun          |
| 1345  | Ballia         | Shiva          |
| 13457 | Ballia         | Shiva          |
| 134588 | New City Ballia | Shiva Updated with Merge |
+-----+-----+-----+
7 rows in set (0.00 sec)

```

Delete the record in the table using delete and remove

delete() Method

- **Usage:** session.delete(entity)
- **Description:** This method was used to remove a persistent instance from the datastore. The argument could be an instance associated with the receiving Session or a transient instance with an identifier associated with an existing persistent state.
- **Deprecation:** The delete() method has been deprecated since Hibernate version 6.0. It is recommended to use the remove() method instead.
- **Cascading:** This operation cascades to associated instances if the association is mapped with cascade="delete".

remove() Method

- **Usage:** session.remove(entity)
- **Description:** This method is aligned with the JPA specification and is used to remove the given entity instance from the database.
- **Cascading:** Similar to delete(), it supports cascading if configured.
- **Recommendation:** Since Hibernate 6.0, remove() is the preferred method for deleting entities.

```
package com.telusko.app;

import org.hibernate.HibernateException;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.cfg.Configuration;

import com.telusko.model.Student;

public class LaunchDelete {
```

```

public static void main(String[] args) {
    // TODO Auto-generated method stub

    SessionFactory sessionFactory = new Configuration().configure().

addAnnotatedClass(Student.class).buildSessionFactory();
    Session session=null;
    Transaction transaction=null;
    boolean flag=false;

    try
    {
        session=sessionFactory.openSession();
        transaction=session.beginTransaction();
        Student st=new Student();
        st.setSid(3);
        st.setName("Harsh");
        st.setScity("Jaipur");
// session.persist(st);

        session.delete(st);
// session.remove(st);

        flag=true;

    }
    catch(HibernateException e)
    {
        e.printStackTrace();
    }
    catch(Exception e)
    {
        e.printStackTrace();
    }
}

```

```
    }  
    finally  
    {  
        if(flag==true)  
            transaction.commit();  
        else  
            transaction.rollback();  
  
        session.close();  
        sessionFactory.close();  
    }  
}  
  
}
```

Output:

```
mysql> select * from studenttable;  
+-----+-----+-----+  
| SID | SCITY | SNAME |  
+-----+-----+-----+  
| 3 | Jaipur | Harsh |  
+-----+-----+-----+  
1 row in set (0.00 sec)  
  
mysql> select * from studenttable;  
Empty set (0.00 sec)
```

Zero Param Constructor for Hibernate

Hibernate:

```
select
    null,
    s1_0.SNAME,
    s1_0.SCITY
from
    StudentTable s1_0
where
    s1_0.SID=?
```

Hibernate:

```
delete
from
    StudentTable
where
    SID=?
```


Maven Project Update

By default when using **eclipse as IDE**, Hibernate or Maven projects may use JDK 1.5 if no specific JDK version is configured. However, you can update the JDK version by adding properties in the pom.xml file.

1. Default JDK Version (1.5):

- When Maven does not explicitly define the JDK version, it defaults to JDK 1.5.
- This can cause issues with newer features available in later versions of Java.

2. Updating the JDK Version:

- To update the JDK version for compiling your project, you need to specify the desired JDK version in the pom.xml file.
- You can do this by adding properties for maven.compiler.source and maven.compiler.target.

3. Properties to Update JDK Version:

To update the JDK version, add the following properties to your pom.xml:

```
<properties>
    <maven.compiler.source>14</maven.compiler.source> <!-- Use JDK 14
for compiling -->
    <maven.compiler.target>14</maven.compiler.target> <!-- Target JDK 14
-->
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
<!-- Ensures UTF-8 encoding -->
</properties>
```

4. Example pom.xml:

Here's an example pom.xml file with updated properties for JDK 14, along with Hibernate and MySQL dependencies:

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>
  <artifactId>HibernateFirst</artifactId>
  <version>0.1.0-SNAPSHOT</version>
  <name>HibernateFirst</name>

  <properties>
    <maven.compiler.source>14</maven.compiler.source> <!-- Updated
JDK version -->
    <maven.compiler.target>14</maven.compiler.target> <!-- Updated JDK
version -->
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  <!-- Encoding -->
  </properties>

  <dependencies>
    <!-- Hibernate Core Dependency -->
    <dependency>
      <groupId>org.hibernate.orm</groupId>
      <artifactId>hibernate-core</artifactId>
      <version>6.3.1.Final</version>
    </dependency>

    <!-- MySQL Connector Dependency -->
    <dependency>
      <groupId>mysql</groupId>
      <artifactId>mysql-connector-java</artifactId>
      <version>8.0.30</version>
    </dependency>
```

```
        </dependencies>  
</project>
```

Summary:

- **Default JDK:** If not set, Maven projects use JDK 1.5.
- **Update JDK:** Use `maven.compiler.source` and `maven.compiler.target` properties to specify a newer JDK version (e.g., JDK 14).
- **Example:** The `pom.xml` above updates the project to use JDK 14, with dependencies for Hibernate and MySQL.

Selective Insertion with @Transient Annotation

The @Transient annotation in Java is used in JPA/Hibernate to mark a field of a class that should **not** be persisted into the database. Fields marked with @Transient are not included in any insert or update operations in the database. It is useful when you want certain fields to be ignored by JPA, either because they are calculated values or are used for business logic only.

Key Points:

1. Ignore during Persistence:

- Fields marked with @Transient will **not be saved** to the database.
- They will also **not be fetched** from the database.

2. Non-persistent Field:

- These fields are used only in the Java object and **not part of the database schema**.
- JPA ignores the annotated field for insert, update, or query operations.

3. Common Use Cases:

- Fields used for business logic.
- Calculated or derived fields (e.g., age based on dateOfBirth).
- Temporary data, which doesn't need to be stored.

4. Example:

Student.java

```
import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
import jakarta.persistence.Transient;
```

```
@Entity
```

```
@Table(name="StudentTable")
```

```
public class Student {
```

```
    @Id
```

```
@Column(name="sid")  
    private int id;
```

```
@Column(name="sname")  
    private String name;
```

```
@Transient  
    private String city;
```

```
public Student() {  
    System.out.println("Zero param constructor");  
}
```

```
public int getId() {  
    return id;  
}
```

```
public void setId(int id) {  
    this.id = id;  
}
```

```
public String getName() {  
    return name;  
}
```

```
public void setName(String name) {  
    this.name = name;  
}
```

```
public String getCity() {  
    return city;  
}
```

```
public void setCity(String city) {  
    this.city = city;  
}
```

```
}
```

Main.java

Refer Insertion Notes

Output:

```
mysql> use telusko_db;
Database changed
mysql> show tables;
+-----+
| Tables_in_telusko_db |
+-----+
| personinfo           |
| studenttable         |
+-----+
2 rows in set (0.07 sec)

mysql> select * from studenttable
-> ;
+-----+-----+
| sid  | sname |
+-----+-----+
| 13457 | Shiva |
+-----+-----+
1 row in set (0.00 sec)
```

In the above example, the bonus field is annotated with `@Transient` and will not be saved in the database, even though it's part of the Java object.

5. Difference from transient keyword:

- **@Transient:** Specifically for JPA (persistence framework) to ignore the field during database operations.
- **transient keyword:** Used in Java for fields that should not be serialized (during Java object serialization).

6. Important:

- `@Transient` fields can still be used in the application, but they will not be part of the database entity mapping.

Data Retrieval with get method

In Hibernate, the `get()` method is used to retrieve an entity from the database based on its primary key. It fetches the data and converts it into a Java object automatically. Here's a detailed explanation of how data retrieval works using Hibernate's `get()` method:

1. Entity Definition (Student Class)

First, we define the entity class that is mapped to the database table. In this case, the class `Student` is mapped to the `StudentTable` in the database.

```
import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name="StudentTable")
public class Student {
    @Id
    @Column(name="sid")
    private int id;

    @Column(name="sname")
    private String name;

    @Column(name="scity")
    private String city;

    public Student() {
        System.out.println("Zero param constructor");
    }
}
```

```

    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public String getCity() {
        return city;
    }
    public void setCity(String city) {
        this.city = city;
    }
}

```

2. Data Flow During Retrieval

Here's how data retrieval happens step-by-step using get():

- **Step 1:** The get() method is called on the Session object. For example:

```
Student student = session.get(Student.class, 1);
```

- **Step 2:** Hibernate (ORM) takes care of the following:
 - It reads the **mapping details** from annotations (@Entity, @Table, @Column) to understand which table and columns to target.
 - It uses the configuration settings for the database connection.
- **Step 3:** Hibernate translates the get() method call into a SQL query using JDBC:

```
SELECT * FROM StudentTable WHERE sid=1;
```


- **Step 4: JDBC** executes the query on the database, retrieving the record.
- **Step 5:** Hibernate processes the result set. It finds the matching fields from the result set (sid, sname, scity) and maps them to the corresponding fields in the Student class.
- **Step 6:** Hibernate automatically creates an instance of the Student class and populates the fields with the data from the result set.
- **Step 7:** Hibernate calls the **zero-parameter constructor** (Student()) of the entity class and sets the values using setter methods (setId(), setName(), setCity()).

3. ResultSet to Object Mapping

Internally, Hibernate uses **ResultSet** from JDBC to get the data from the database. For each record in the ResultSet:

- It **calls the constructor** of the Student class to create an object.
- It **sets the values** of each field (id, name, city) using reflection and setter methods.

4. Main Class to Demonstrate Retrieval

Here's a simple main class that demonstrates how to retrieve data using Hibernate's get() method.

```
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;

public class Main {
    public static void main(String[] args) {
        // Configure Hibernate
        SessionFactory factory = new
Configuration().configure("hibernate.cfg.xml")
        .addAnnotatedClass(Student.class).buildSessionFactory();

        // Open session
```

```

Session session = factory.openSession();

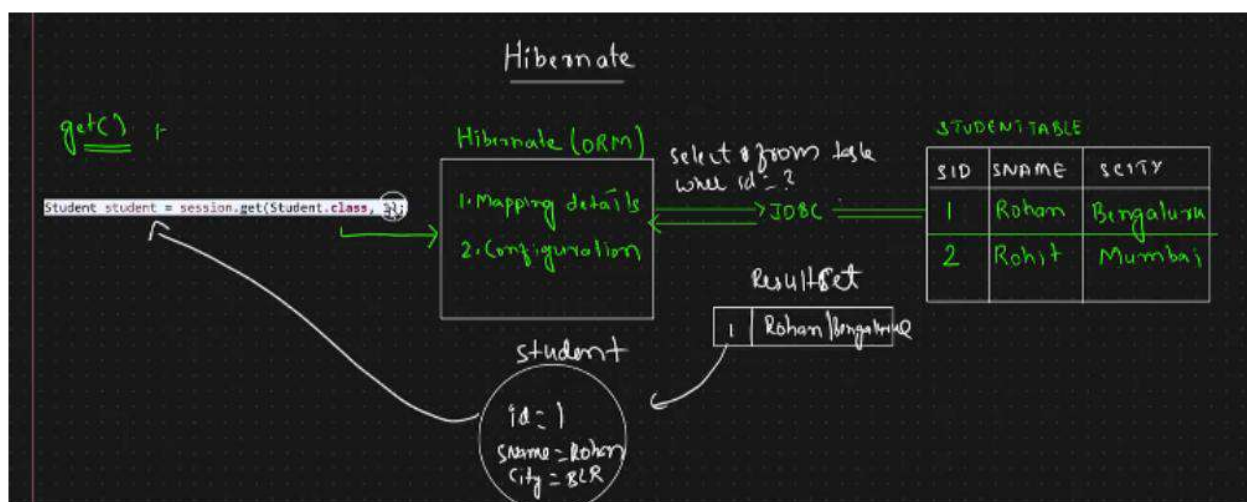
try {

    // Retrieve student with ID 1
    Student student = session.get(Student.class, 1);

    // Display the student details
    System.out.println("ID: " + student.getId());
    System.out.println("Name: " + student.getName());
    System.out.println("City: " + student.getCity());

} finally {
    // Close session
    session.close();
    factory.close();
}
}

```



5. How ORM Works Behind the Scenes

- Hibernate simplifies the database interaction by mapping Java objects (entities) to database tables.
- The `get()` method abstracts away the complexity of SQL queries and JDBC code.
- **Automatic Object Creation:** Hibernate takes care of object creation from database records. You don't have to manually write code to parse the `ResultSet`.
- **Zero-Parameter Constructor:** Hibernate requires a no-arg constructor to create instances of the entity class.

6. Advantages of `get()` Method

- **Simple API:** The `get()` method is simple to use and returns null if no entity is found.
- **Eager Loading:** The `get()` method fetches the object immediately and returns a fully initialized entity.
- **Convenience:** Hibernate handles mapping and data retrieval without manual SQL queries.

Lazy Loading vs Eager Loading in Hibernate

In Hibernate, **lazy loading** and **eager loading** control how related entities are fetched from the database. The `get()`, `load()`, and `getReference()` methods play crucial roles in these processes, especially when dealing with proxy objects, transient objects, and the timing of data retrieval.

1. `get()` Method

- **Immediate Fetch** (Eager Loading): The `get()` method fetches the object from the database immediately, returning a fully initialized object.
- **Usage:** It is typically used when you need the actual object right away.
- **Return Type:** Returns null if no record is found.

Example:

```
Session session = sessionFactory.openSession();
Student student = session.get(Student.class, 1); // Eagerly loads the object
System.out.println("ID: " + student.getId());
System.out.println("Name: " + student.getName());
System.out.println("City: " + student.getCity());
```

- **Behavior:** The `get()` method sends an immediate SQL SELECT query to the database to retrieve the record. The object is fully initialized when fetched.
- **Use Case:** Use `get()` when you need immediate access to the object, like when displaying student details.

2. `load()` Method

- **Lazy Loading:** The `load()` method doesn't hit the database immediately. Instead, it returns a proxy object, and the actual database query is executed when you access a property of the entity.

- **Usage:** It is used when you might or might not need to access the object fields.
- **Return Type:** Throws an exception if the object is not found.

Example:

```
Session session = sessionFactory.openSession();
Student student = session.get(Student.class, 1); // Eagerly loads the object
System.out.println("ID: " + student.getId());
System.out.println("Name: " + student.getName());
System.out.println("City: " + student.getCity());
```

- **Behavior:** load() returns a proxy object (a placeholder object). The actual query to the database is triggered only when a method like getName() or getCity() is called.
- **Proxy Object:** If you try to print the proxy object directly, it won't show the actual values until a method is accessed. When accessing the object's properties, Hibernate initializes the object by executing the query.
- **Use Case:** Use load() when you might not need the object immediately or in scenarios where performance optimization is needed by delaying the database call.

3. getReference() Method

- **Lazy Loading:** The getReference() method replaces the deprecated load() method in newer versions of Hibernate, maintaining the same lazy loading behavior by returning a proxy object.
- The actual query is executed only when the properties of the object are accessed.
- **Usage:** Typically used in entity relationships.
- **Return Type:** Throws an exception if the object is not found.

Example:

```
Session session = sessionFactory.openSession();
```

```
Student student = session.getReference(Student.class, 1); // Proxy object
System.out.println("ID: " + student.getId());           // Executes query
```

- **Behavior:** Works similarly to `load()` by returning a proxy object. It triggers the database query only when the object properties are accessed.

4. Proxy Object Behavior

A **proxy object** is a placeholder object returned by Hibernate when using lazy loading (`load()` or `getReference()`). The proxy object doesn't contain the actual data; instead, the database is queried when an attribute (like `getName()` or `getCity()`) is accessed.

- **Accessing Proxy Object:** If the entity's properties (e.g., `getName()`, `getCity()`) are accessed, Hibernate triggers a database call to fetch the full object data. If the entity is **not found**, Hibernate throws an exception.
- **Proxy Object Example:**

```
Student student = session.load(Student.class, 1); // Returns a proxy object
System.out.println("ID: " + student.getId());     // Triggers actual database call
```

5. Transient Object

A **transient object** in Hibernate is an object that has not been associated with any database session. It doesn't have a corresponding row in the database.

- **Behavior with Transient Objects:** If a transient object is used with methods like `get()` or `load()`, Hibernate will not find it in the session or database, which might lead to exceptions.

6. Key Differences Between `get()`, `load()`, and `getReference()`

- **`get()`:** Always hits the database immediately. Returns null if the object is not found.
- **`load()`:** Returns a proxy object. Throws an exception if the object is not found.

- **getReference():** Similar to load(), returns a proxy object and throws an exception if the object is not found.

7. Example: Handling Proxy Object and Lazy Loading

```
Session session = sessionFactory.openSession();
try {
    // Using get()
    Student student = session.get(Student.class, 1);
    if (student != null) {
        System.out.println("ID: " + student.getId());
        System.out.println("Name: " + student.getName());
        System.out.println("City: " + student.getCity());
    } else {
        System.out.println("No data found with the given ID");
    }

    // Using load()
    //Student proxyStudent = session.load(Student.class, 2); // Lazy loading
    Student proxyStudent = session.getReference(Student.class, 2);
    System.out.println("Accessing proxy object: " + proxyStudent.getName());
} catch (HibernateException e) {
    e.printStackTrace();
} finally {
    session.close();
}
```

Summary of Key Points:

- **get():** Immediate data retrieval (eager loading), returns null if not found.
- **load() and getReference():** Lazy loading, returns a proxy object, triggers data fetch only when properties are accessed, and throws exceptions if not found.

- **Proxy Object:** Used in lazy loading to delay the database query. Hibernate provides an actual instance only when properties are accessed.
- **Transient Objects:** Objects not yet associated with the session, potentially leading to exceptions if accessed in certain ways.
- **Lazy Initialization Exception:** Be careful when closing the session before accessing lazy-loaded properties.